



Data Layer (`app/data/`)

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Data Layer (app/data/)	1
Overview	3
Module Index	4
models.py — Domain Dataclasses	5
source_router.py — SEC vs YFinance Routing	6
segment_aliases.py — Geographic Segment Canonicalization	6
screening_config.py — Screening Constants	7
screening_service.py — Screening Execution Engine	8
repository.py — Repository Catalog	9
User Persistence Services	11
Earnings Alert Services	12
Forecasting Pipeline	12
live_earnings_transcript_store.py	13
dummy_data.py — Legacy Stub	13
Design Conventions	13
Page → Data Module Map	14

Application: Market Data Portal (MDP)

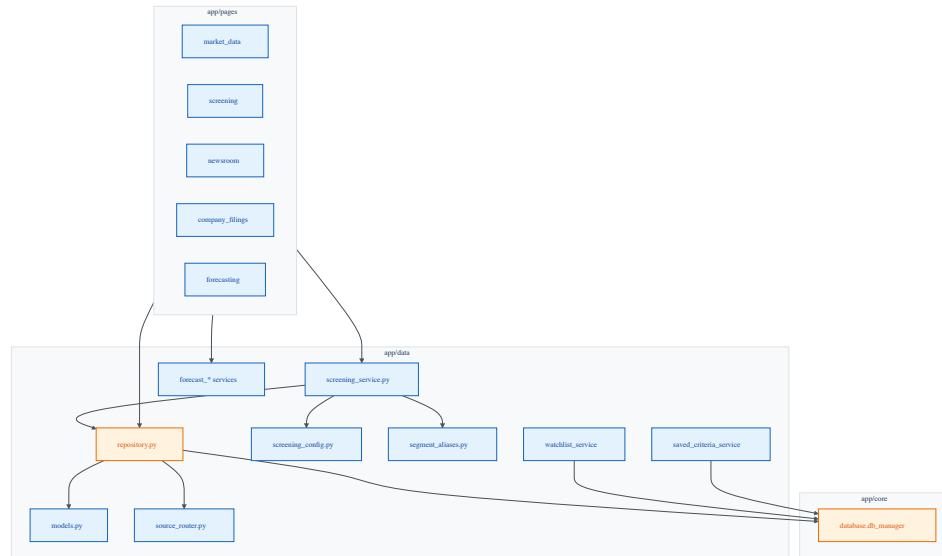
Stack: SQLAlchemy 2 via `core.database.db_manager`, Streamlit `@st.cache_data`

Scope: Models, repositories, screening engine, source routing, segment aliases, user persistence, forecasting pipeline

Overview

`app/data/` is the domain and data-access layer between Streamlit pages and Azure MySQL. All persistent tables use the `coreiq_` prefix. Reads prefer `db_manager.execute_query_readonly` (AUTOCOMMIT, 1 RTT on Azure).

app/data/ is the domain and data-access layer between Streamlit pages and Azure MySQL. All persistent tables use the coreiq prefix. Reads prefer dbmanager.executequeryreadonly (AUT



Module Index

File	Role
<code>models.py</code>	Dataclasses + fiscal calendar helpers
<code>repository.py</code>	19 repository classes — primary Database (DB) read layer (~11k lines)
<code>screening_config.py</code>	Screening constants (zero DB calls)
<code>screening_service.py</code>	Screening SQL/pandas engine
<code>source_router.py</code>	U.S. Securities and Exchange Commission (SEC) vs YFinance per ticker
<code>segment_aliases.py</code>	Geographic segment canonicalization
<code>watchlist_service.py</code>	Watchlist Create, Read, Update, Delete (CRUD) + Access Control List (ACL)
<code>saved_criteria_service.py</code>	Saved screening criteria + ACL
<code>portal_users_service.py</code>	Internal user registry for ACL pickers
<code>earnings_alert_service.py</code>	Earnings email alert prefs + dedupe
<code>earnings_alert_store.py</code>	Re-export shim → <code>earnings_alert_service</code>
<code>forecast_store.py</code>	<code>coreiq_model_forecasts</code> read/write cache
<code>forecast_admin_service.py</code>	Admin bulk forecast sync
<code>forecast_refresh_service.py</code>	Forecast refresh popup + email
<code>revenue_forecast_service.py</code>	Revenue Estimates / Forecasting page data
<code>revenue_estimates_service.py</code>	Alias: <code>RevenueEstimatesService</code> = <code>RevenueForecastService</code>
<code>live_earnings_transcript_store.py</code>	Live transcript upsert
<code>dummy_data.py</code>	Legacy stub — incompatible with current <code>models.py</code>
<code>__init__.py</code>	Package marker

models.py — Domain Dataclasses

Dataclass catalog

Class	Source / usage
<code>TickerSentiment</code>	Per-ticker sentiment on news articles
<code>NewsArticle</code>	<code>coreiq_av_market_news_sentiment</code> (+ <code>source_tag</code> : <code>av</code> <code>yf</code>)
<code>Company</code>	<code>coreiq_companies</code>
<code>IncomeStatementLineItem</code>	Display row for income statement grid
<code>FiscalPeriod</code>	Column header (annual/quarterly, Calendar Quarter (CQ)/Fiscal Quarter (FQ), estimate/forecast flags)
<code>IncomeStatementData</code>	Company + periods + line items bundle
<code>CompanyOverview</code>	Alpha Vantage (AV)/Yahoo Finance (YF) company overview payload
<code>BalanceSheetLineItem</code> / <code>BalanceSheetData</code>	Balance sheet display
<code>CashFlowLineItem</code> / <code>CashFlowData</code>	Cash flow display
<code>EarningsCall</code>	<code>coreiq_av_earnings_call_transcripts</code>
<code>FilingMetricResult</code>	<code>coreiq_filing_metrics_v2</code> / v5 filing metrics

Key properties

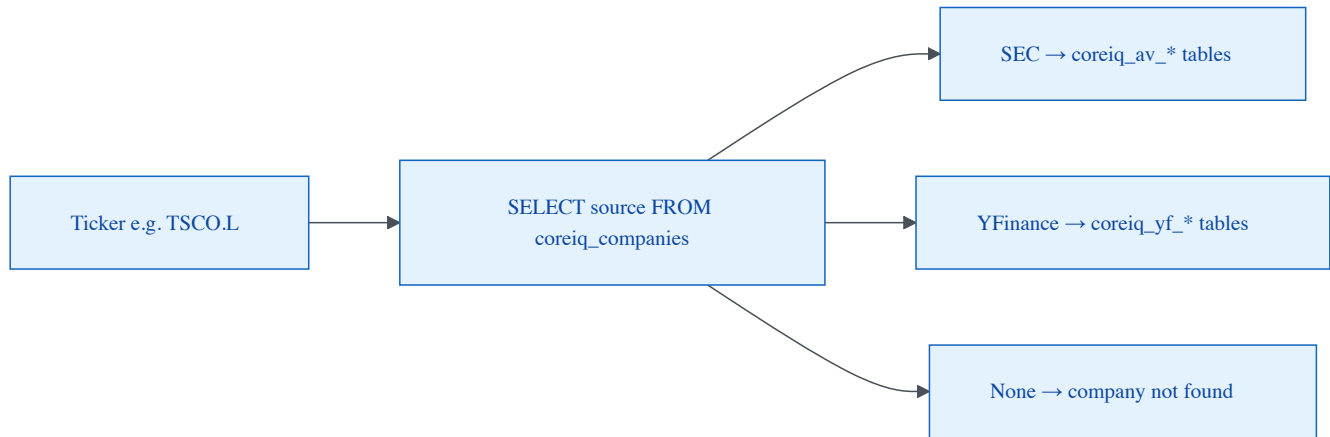
- **NewsArticle** : `formatted_date` , `company_tickers`
- **Company** / **CompanyOverview** : `display_name` , `formatted_market_cap` , `website_display`
- **EarningsCall** : `display_title` , `formatted_quarter` , `speaker_sections` (transcript parser)
- **FilingMetricResult** : `formatted_value` , `display_label` , `display_statement_type`

Fiscal calendar functions

Function	Returns
<code>get_calendar_quarter(month)</code>	CQ1-4
<code>get_fiscal_quarter(month, fiscal_year_end_month)</code>	FQ1-4
<code>parse_fiscal_year_end(str)</code>	Month 1-12
<code>FiscalPeriod.from_date(dt, period_type, fiscal_year_end)</code>	Labeled period column

source_router.py — SEC vs YFinance Routing

'source_router.py' — SEC vs YFinance Routing



Function	Returns	Cache
<code>get_company_source(ticker)</code>	<code>'SEC' 'YFinance' None</code>	<code>@st.cache_data(ttl=600)</code>
<code>is_sec_company(ticker)</code>	<code>bool</code>	
<code>is_yf_company(ticker)</code>	<code>bool</code>	

Table: `coreiq_companies.source`

Composite tickers: `TSCO.L` resolved via `exchange_acronym='L'`

Nearly every financial repository branches on `get_company_source(ticker)`.

segment_aliases.py — Geographic Segment Canonicalization

Purpose

Conservative US-only geographic grouping for screening segment filters.

Constants

Constant	Role
<code>GEO_CANONICAL_GROUPS</code>	United States, Canada, UK, North America, Non-US, Americas
<code>GEO_ALIAS_TO_CANONICAL</code>	Case-insensitive raw label → canonical
<code>GEO_SUPPRESSED_DROPDOWN_LABELS</code>	Hide duplicate aliases in User Interface (UI)
<code>GEO_PENSION_SKIP_LABELS</code>	Exclude pension-plan contamination
<code>GEO_KNOWN_BIZ_LABELS</code>	Business labels incorrectly in geo presets

Functions

Function	Role
<code>canonicalize_geo_label(raw)</code>	Single label normalization
<code>expand_geo_canonical_segments(selected)</code>	Canonical → all raw aliases for Structured Query Language (SQL) <code>IN</code>
<code>is_geo_pension_label()</code> / <code>is_geo_known_biz_label()</code>	Filter guards

Consumers: `screening_service.py`, `screening.py`, `repository.py` (`SegmentDataRepository`)

`screening_config.py` — Screening Constants

Rule: Zero DB calls. All constants for UI and SQL generation.

Core constants

Constant	Meaning
<code>OPERATORS</code> / <code>OPERATOR_SQL</code>	UI operators → SQL (<code>Between</code> handled separately)
<code>PERIOD_TYPES</code>	<code>Fiscal Year (FY)</code> , <code>CQ</code> , <code>FQ</code>
<code>TRAILING_QUARTERS</code> (<code>TQ</code>)	Display-only last N quarters (4/8/12, default 8)
<code>DB_SCALE</code>	<code>1_000_000</code> — UI \$mm ↔ raw DB dollars
<code>SCREENING_YEARS</code>	Historical FY range (current year → 2015)
<code>FORWARD_SCREENING_YEARS</code>	Includes future FY up to current+5

`STATEMENT_CONFIG`

Maps statement types to DB tables/columns:

Statement type	Data source
Income Statement, Balance Sheet, Cash Flow	AV + YF financial tables
Key Stats, Ratios	Repository tabular output (<code>metric_key</code>)
Business/Geographical Segments	<code>coreiq_filing_metrics_v5</code>
Estimates	AV wide + YF long earnings estimates
Forecasting	<code>coreiq_model_forecasts</code> <code>model_key</code>

News & Key Developments

- `NEWSDEV_TOPICS` — 15 AV NLP topic keys
- `NEWSDEV_TIMEFRAMES` — 7/30/90/365 days
- `KEYDEV_CATEGORIES` — 12-category CIQ taxonomy
- `KEYDEV_CATEGORIES_ALL` — 24 Excel-style DB categories
- `KEYDEV_TIMEFRAMES` — 1 day through All History

Helpers

- `get_metric_labels(statement)`, `get_metric_info(statement, label)`

screening_service.py — Screening Execution Engine

Rule: No Streamlit UI. Returns pandas DataFrames + debug metadata.

Pipeline

Screening narrows the company universe through progressive criterion filters until the working set matches all active rules.

Screening narrows the company universe through progressive criterion filters until the working set matches all active rules.



Criterion applicators

Function	Criterion type
<code>apply_industry_criteria()</code>	Industry multi-select
<code>apply_geography_criteria()</code>	Country filter
<code>apply_financial_criteria()</code>	FY/CQ/FQ financial metrics
<code>apply_key_stats_criteria()</code> / <code>apply_ratios_criteria()</code>	Tabular market data
<code>apply_estimates_criteria()</code> / <code>apply_forecast_criteria()</code>	Forward-looking
<code>apply_segment_statement_criteria()</code>	Business/Geo segment statements
<code>apply_keydevs_criteria()</code>	Key developments events
<code>apply_biz_segments_criteria()</code> / <code>apply_geo_segments_criteria()</code>	Legacy segment filters

Segment cache tables

- `coreiq_screening_segment_member_cache`
- `coreiq_screening_segment_values_cache`

Functions: `build_segment_values_cache()`, `read_segment_member_options_cache()`, `get_segment_member_options()`, etc.

Orchestration

Function	Role
<code>recompute_working_set(active_criteria, criterion_cache, allowed_tickers, allowed_members)</code>	Main pipeline; per-criterion fingerprint cache
<code>filter_universe_to_members()</code>	<code>(ticker, company_name)</code> composite key filter
<code>criteria_stack_fingerprint()</code>	Cache key for full stack

Scale rule: User enters \$mm; compared against raw DB dollars × `DB_SCALE` (1e6).

`repository.py` — Repository Catalog

Central read layer. All classes use static methods, `db_manager`, `source_router`, and `@st.cache_data`.

Repository classes

#	Class	Primary tables	Key methods
1	CompanyRepository	coreiq_companies	get_companies_map , get_companies_rows , get_company_by_ticker , get_all_sectors
2	IncomeStatementRepository	coreiq_av/yf_financials_income_statement	get_income_statement_data , get_date_range , get_reported_currency
3	NewsRepository	coreiq_av/yf_market_news_sentiment	get_articles , get_yf_articles , get_news_tickers , get_sectors
4	CompanyOverviewRepository	coreiq_av/yf_company_overview	get_company_overview , company_exists
5	EarningsCallRepository	coreiq_av_earnings_call_transcripts	get_earnings_calls , search_transcripts_fulltext , get_non_sec_transcript_*
6	BalanceSheetRepository	AV/YF balance sheet	get_balance_sheet_data
7	CashFlowRepository	AV/YF cash flow	get_cash_flow_data
8	KeyStatsRepository	Computed from statements + overview	get_key_stats_data , extract_metric_value , get_estimated_data
9	AnalystEstimatesRepository	AV/YF earnings estimates	get_estimates_data
10	ModelForecastsRepository	coreiq_model_forecasts	get_forecasts_data (ensemble + scenarios)
11	RatiosRepository	Computed in Python	get_ratios_data (liquidity, leverage, margins, ROA/ROE, P/E)
12	ForexRepository	coreiq_av_forex_daily	get_conversion_rate , get_conversion_rates_bulk
13	StockQuoteRepository	AV/YF time series daily	get_latest_quote , get_price_history , get_market_cap_chart_data
14	FilingMetricRepository	coreiq_filing_metrics_v5	search_with_llm_fallback , get_header_metadata , SYNONYM_MAP
15	AVFinancialsEarningsRepository	coreiq_av_financials_earnings	get_earnings_date_pair_for_fiscal_quarter_e
16	EarningsCalendarRepository	NASDAQ + YF calendar, IR websites	get_calendar_events , get_transcript_for_calendar_event , get_earnings_alert_preferences
17	SegmentDataRepository	coreiq_filing_metrics_v5 + edgartools	get_segment_data , get_square_footage_data
18	RatingsDataRepository	Filing metrics (credit_rating, store_count)	get_ratings_data
19	ExecutiveCompensationRepository	coreiq_executives_compensation , YF overview	get_sec_latest_compensation , get_yf_latest_compensation

Module-level helpers

- normalize_title_for_dedupe(title) — Unicode-safe news dedupe
- _get_fiscal_year_end_cached(ticker) — 30 min cache
- warmup_ec_caches() — Earnings calendar cache warmup (called from main.py)

Caching TTL conventions

TTL	Use
60s	Earnings calls
300s	Financial statements, quotes, segments
600s	Company map, overview, source router
3600s	Companies rows, fiscal year end, news reference
43200s	News articles list

User Persistence Services

watchlist_service.py

Tables: coreiq_watchlists , coreiq_watchlist_access , coreiq_watchlist_companies

Function	Action
ensure_tables()	Idempotent DDL
create_watchlist()	Insert metadata
get_user_watchlists()	Own + granted (120s cache)
get_watchlist_companies() / get_watchlist_tickers()	Members
add_companies_to_watchlist() / remove_company_from_watchlist()	Membership
grant_watchlist_access() / revoke_watchlist_access()	ACL

Consumers: screening.py , newsroom.py , earnings_calls.py , earnings_calendar.py

saved_criteria_service.py

Tables: coreiq_saved_criteria , coreiq_saved_criteria_access

Function	Action
save_criteria() / get_user_criteria() / get_criteria_by_id()	CRUD
update_criteria() / delete_criteria()	Soft delete
grant_access() / revoke_access()	ACL

Consumer: screening.py

portal_users_service.py

Table: coreiq_portal_users — seeded with ~40 Coresight team members

Function	Role
get_all_portal_users(exclude_email)	ACL user-picker dropdown
upsert_portal_user()	Add/update user

Consumers: screening.py , access_management.py

Earnings Alert Services

`earnings_alert_service.py`

Tables:

- `coreiq_earnings_alert_preferences_history` — append-only; latest row = current settings
- `coreiq_earnings_alert_delivery_log` — unique `dedupe_key` per sent email

Function	Role
<code>load_preference(user_email)</code> / <code>save_preference(...)</code>	enabled, days_before, selection_mode, tickers, sectors
<code>list_enabled_preferences()</code>	Users with latest <code>enabled=1</code>
<code>delivery_exists(dedupe_key)</code> / <code>try_insert_delivery()</code>	Idempotent send guard

Consumers: `earnings_calendar.py`, `utils/earnings_alert_dispatcher.py`

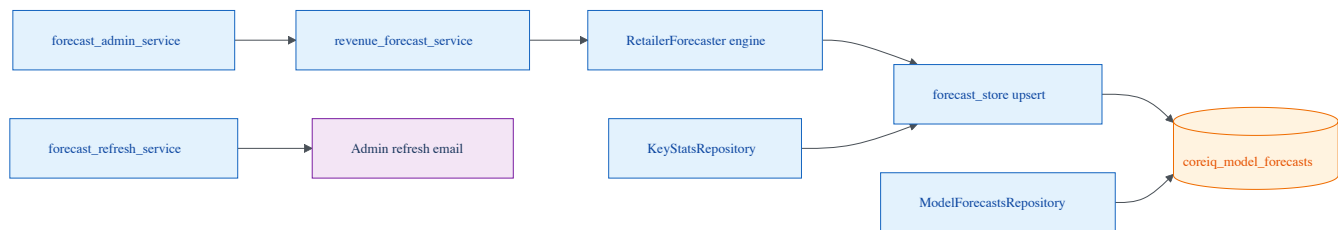
`earnings_alert_store.py`

Re-export shim only — imports from `earnings_alert_service`.

Forecasting Pipeline

Revenue forecasts flow from the forecasting engine into `coreiq_model_forecasts`, with admin sync and Key Stats consumption paths.

Revenue forecasts flow from the forecasting engine into coreiqmodelforecasts, with admin sync and Key Stats consumption paths.



`forecast_store.py`

Function	Role
<code>needs_update(ticker, last_actual_date)</code>	Staleness check (1 RTT)
<code>upsert_forecasts(...)</code>	<code>INSERT ... ON DUPLICATE KEY UPDATE</code>
<code>get_forecasts(ticker, model_key='ensemble')</code>	Read for Key Stats F- columns
<code>delete_forecasts_beyond_horizon(max_periods_ahead=5)</code>	Cleanup

Model keys: `ensemble`, `linear`, `cagr`, `exp_smoothing`, `holt`, `ma_trend`, `weighted_avg`

Scenario keys: `scenario_pessimistic`, `scenario_baseline`, `scenario_optimistic`

`revenue_forecast_service.py`

Class: `RevenueForecastService`

Method	Description
<code>get_companies()</code>	Companies with ≥3 annual revenue points
<code>get_company_dashboard(ticker, periods=5)</code>	Actuals, backtest, forecast, summary

Engine: `utils.retailer_forecaster.RetailerForecaster`

Fast path: Reconstructs forecast from `coreiq_model_forecasts` when `needs_update()` is false

`forecast_admin_service.py`

Function	Role
<code>is_forecast_admin(email)</code>	Hardcoded admin allowlist (4 emails)
<code>sync_forecast_for_ticker(display_ticker, force, periods=5)</code>	Single-ticker sync
<code>sync_all_eligible(force, progress_callback)</code>	Bulk sync
<code>clear_revenue_forecast_caches()</code>	Clear Streamlit caches

`forecast_refresh_service.py`

Function	Role
<code>get_refresh_table_data(period_type)</code>	All-ticker refresh status table
<code>send_model_refresh_email()</code>	HyperText Markup Language (HTML) Simple Mail Transfer Protocol (SMTP) notification to admins
<code>validate_q4_reporting_dates()</code>	Q4 date consistency audit
<code>ensure_forecast_columns()</code>	DDL migrations

`live_earnings_transcript_store.py`

Table: `coreiq_earnings_call_live_transcripts`

Source tag: `live_streamlit`

Unique key: `(source, ticker, quarter)` where `quarter = 2024Q1`

Function	Role
<code>export_period_key(fiscal_year, quarter)</code>	<code>2024Q1</code> format
<code>build_earnings_transcript_payload()</code>	JSON transcript structure
<code>upsert_live_transcript_row()</code>	INSERT ... ON DUPLICATE KEY UPDATE

Consumer: `live_earnings_transcript.py`

`dummy_data.py` — Legacy Stub

Status: Broken — imports types (`SEC filing` , `MarketData`) that no longer exist in `models.py` . Production uses `repository.py` .

Design Conventions

1. **Logging:** `utils.server_logger` only (`log_structured_error` , `log_db_timing`)

2. **No UI in services:** `screening_service.py` is pure data; pages own Streamlit
3. **ACL:** Watchlists and saved criteria enforce server-side grants
4. **Idempotent DDL:** Services use `CREATE TABLE IF NOT EXISTS` + threading locks
5. **Source routing:** SEC → `coreiq_av_*` wide tables; YFinance → `coreiq_yf_*` long/pivot; composite tickers use `yf_symbol` or `TICKER.EXCHANGE`

Page → Data Module Map

Page	Primary data imports
<code>market_data.py</code>	<code>repository.*</code> , <code>models</code> , <code>source_router</code>
<code>screening.py</code>	<code>screening_config</code> , <code>screening_service</code> , <code>watchlist_service</code> , <code>saved_criteria_service</code> , <code>portal_users_service</code> , <code>segment_aliases</code>
<code>newsroom.py</code>	<code>repository.NewsRepository</code> , <code>models</code> , <code>watchlist_service</code>
<code>earnings_calls.py</code>	<code>repository.EarningsCallRepository</code> , <code>watchlist_service</code>
<code>company_filings.py</code>	<code>repository.FilingMetricRepository</code>
<code>forecasting.py</code>	<code>revenue_forecast_service</code> , <code>forecast_admin_service</code> , <code>forecast_refresh_service</code>
<code>live_earnings_transcript.py</code>	<code>live_earnings_transcript_store</code>
<code>access_management.py</code>	<code>portal_users_service</code> (via ACL UI)
<code>main.py</code>	<code>repository.warmup_ec_caches</code> , <code>watchlist/saved_criteria/portal/earnings_alert</code> <code>ensure_tables</code>